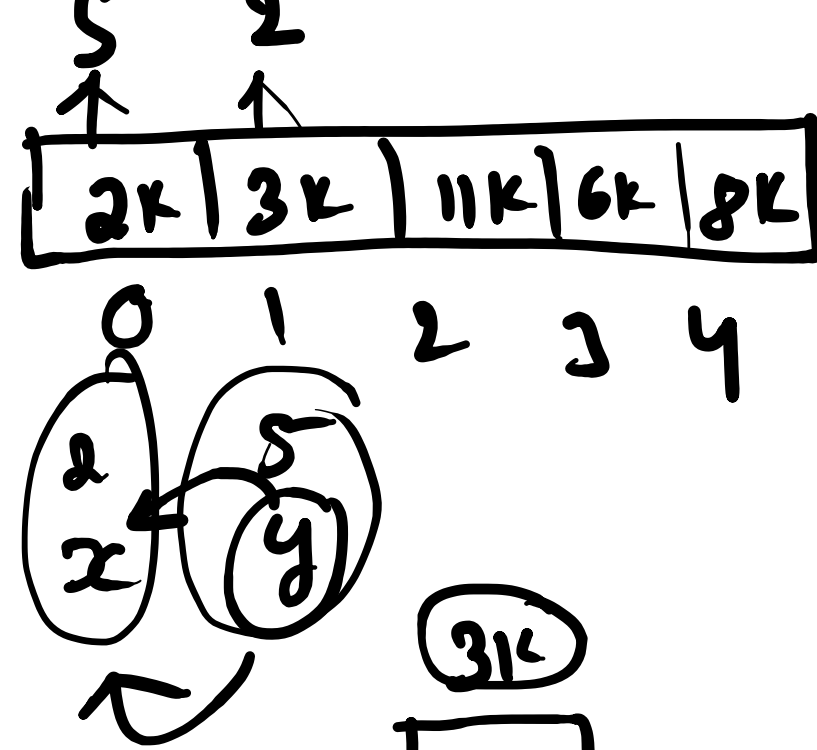


$s_1.compareTo(s_2) > 0 \mid s_1 > s_2$
 $< \mid s_1 < s_2$
 $0 \mid s_1 == s_2$

compareTo

$arr[i].compareTo(arr[i+1]) > 0 \mid arr[i] > arr[i+1]$
 $< \mid arr[i] < arr[i+1]$
 $0 \mid \text{same}$

```
public static <T extends Comparable<T>> void sort(T[] arr) {  
    // TODO Auto-generated method stub  
    for (int turn = 1; turn < arr.length; turn++) {  
        for (int i = 0; i < arr.length - turn; i++) {  
            if (arr[i].compareTo(arr[i + 1]) > 0) {  
                T temp = arr[i];  
                arr[i] = arr[i + 1];  
                arr[i + 1] = temp;  
            }  
        }  
    }  
}
```

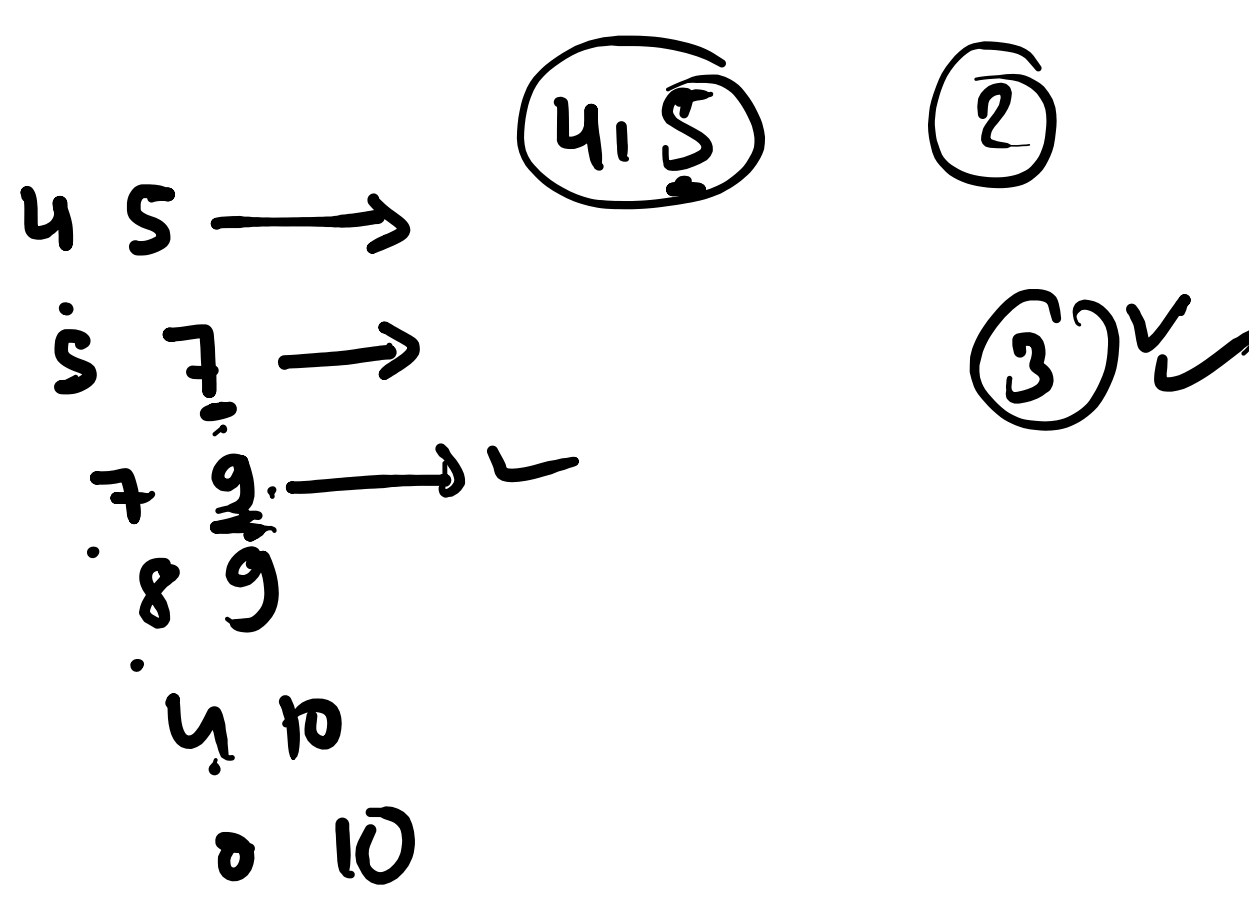
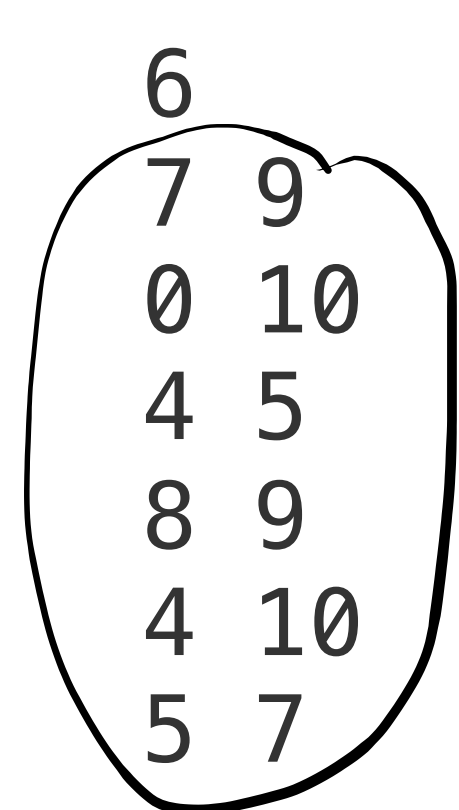
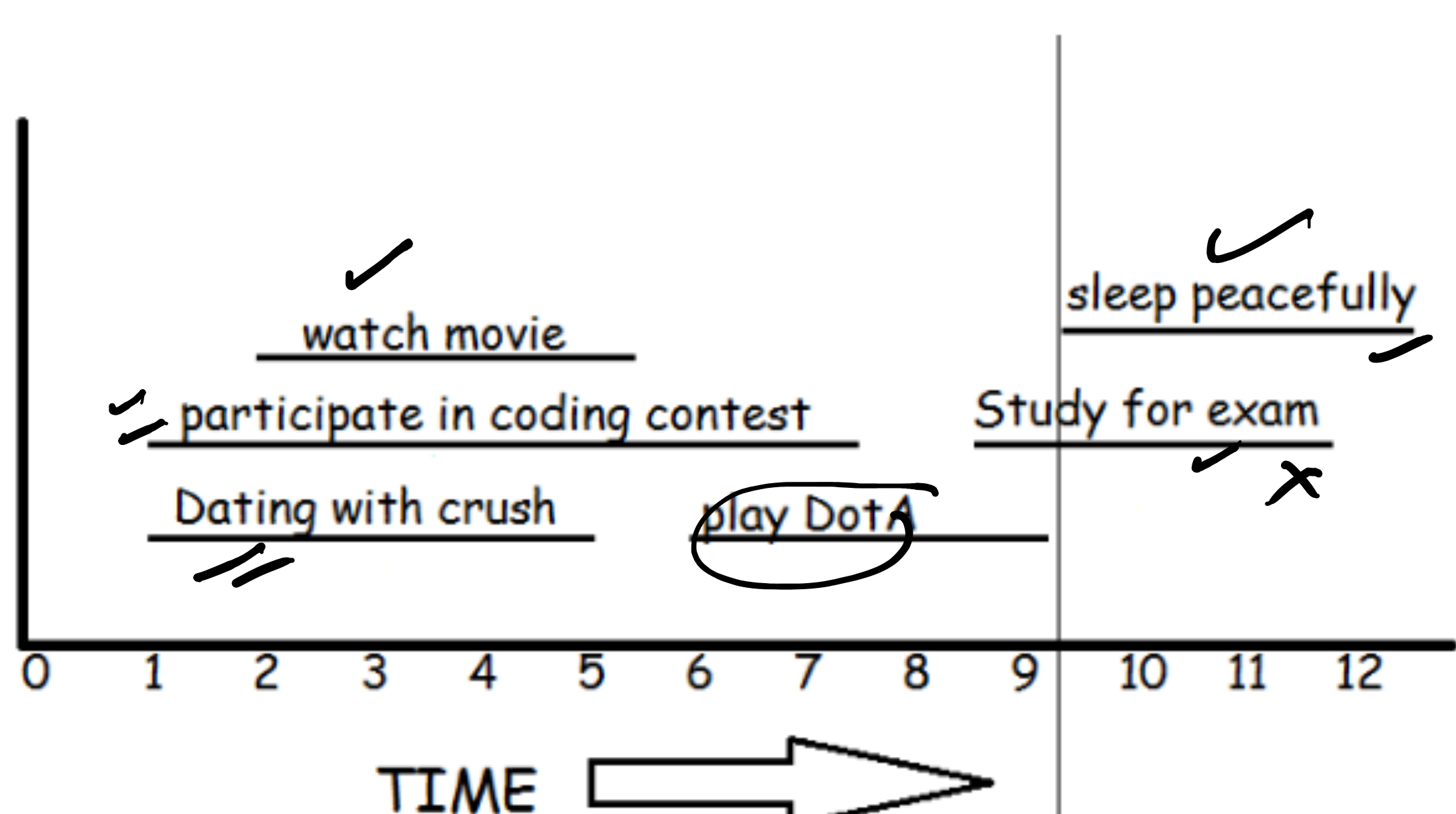


Rank map

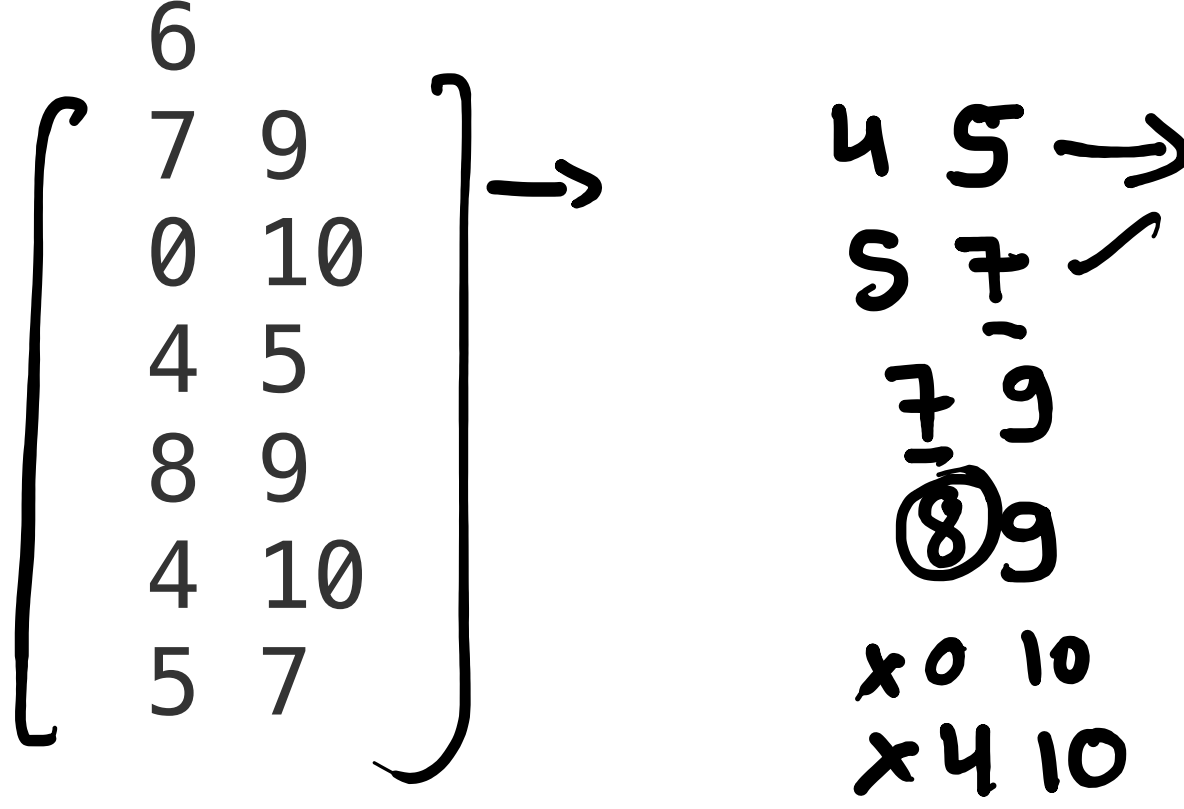
```
public int compareTo(Cars o) {  
    // TODO Auto-generated method stub  
    return 0;  
}
```

$this.P - o.P > 0$
 $o.S - this.S$

$this - o$
Rank
 $o - this$



```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    Pair[] arr = new Pair[n];  
    for (int i = 0; i < arr.length; i++) {  
        int st = sc.nextInt();  
        int et = sc.nextInt();  
        arr[i] = new Pair(st, et);  
    }  
    Arrays.sort(arr, (a, b) -> a.et - b.et);  
    int activitie = 1;  
    int end = arr[0].et;  
    for (int i = 1; i < arr.length; i++) {  
        if (arr[i].st >= end) {  
            activitie++;  
            end = arr[i].et;  
        }  
    }  
    System.out.println(activitie);  
}
```



Priority queue

queue $\rightarrow$ FIFO

heap

$P_i = \frac{C_i - 1}{2}$

BT is CBT and some priority

